

AI Usage & Disclosure Log

This AI usage log documents the parts of the Munch web application where generative AI was most likely used or would be reasonable to disclose. The purpose is not to claim that AI wrote the whole project, but to explain how AI support was used as a coding assistant and how the final code was manually changed to fit the team's preferences, database, UI, and project requirements.

1. Cart System and Combo Customizer

Project Component: Cart System, Add-to-Cart Flow, Combo Set Customizer

AI Tool Used: ChatGPT / Gemini / GitHub Copilot

The Intent

I used AI to help improve the cart system because the add-to-cart function needed to handle different menu items, combo sets, quantity updates, and cart total calculations. I also needed support for a combo customizer where customers could choose different dishes before adding the combo to the cart.

The Interaction (Prompting)

My main prompt asked for help creating a more reliable JavaScript and PHP cart flow. I explained that the project used menu IDs such as `M001`, `M058`, `M059`, and `M060`, and that the menu ID could appear in different places such as `data-menu-id`, `data-id`, `href`, or `onclick`. I also refined the prompt to ask for combo set handling, cart quantity updates, subtotal calculation, and better user feedback.

The Output & Verification

The AI helped suggest code structure for `cart.js`, including helper functions for menu ID detection, price formatting, HTML escaping, combo modal display, `fetch` requests, and cart rendering. The PHP files helped process cart actions and update the displayed cart items. I verified the code by running the project in XAMPP, adding normal menu items, adding combo items, changing quantities, checking subtotal/tax/total values, and confirming that the cart stayed consistent after updates.

Human Refinement

I did not only rename variables. I manually edited the generated logic to match the Munch project's actual coding style and requirements. I changed the menu ID handling, so it worked with my real menu buttons and item IDs. I adjusted the combo set options, combo item mapping, displayed cart text, alert messages, button behaviour, and cart layout to match my preferred user experience. I also connected the frontend JavaScript to my actual PHP filenames and database/session flow instead of using the AI's generic example structure. Some logic was rewritten so the cart worked with the existing Munch pages rather than as a standalone sample.

2. Owner Dashboard, Reports, and Database API

Project Component: Owner Dashboard, Reports, Employee/Menu Management, MySQL API

AI Tool Used: ChatGPT / Gemini / GitHub Copilot

The Intent

I used AI to help connect the owner dashboard to the MySQL database and display real information such as sales, menu data, staff records, reservations, order summaries, and dashboard statistics. The goal was to replace static dashboard data with dynamic data from the project database.

The Interaction (Prompting)

I first asked for help creating a PHP API that could return JSON data from a MySQL database. I refined the prompt by explaining that my project included tables such as `staff`, `owner`, `menu`, `orders`, `ordermenu`, `sales`, and `reservation`. I also asked for JavaScript that could fetch this data and display dashboard cards, tables, report sections, and management pages.

The Output & Verification

The AI helped suggest reusable backend API patterns, SQL queries, JSON responses, and frontend rendering functions for files such as ``api/owner_api.php``, ``ownerData.php``, ``ownerActions.php``, and ``owner-dashboard.js``. I verified the output by importing ``database/munchdb.sql`` into phpMyAdmin, logging in as an owner, opening the dashboard pages, checking browser console and network responses, and comparing displayed values with records in the database.

Human Refinement

I manually edited the AI-assisted code to fit my own database and dashboard preferences. I changed SQL queries to match the actual table names, column names, relationships, and available data in ``munchdb.sql``. I adjusted the JSON response structure, dashboard labels, management table columns, report categories, page sections, and error messages. I also changed file paths, database connection details, session checks, and owner-page behavior so the code worked correctly inside the existing Munch project. Any generated dashboard logic that did not match the project flow was modified or removed.

3. Reservation and Catering Booking System

Project Component: Reservation Page, Catering Package Logic, Deposit Calculation, Reservation Submission

AI Tool Used: ChatGPT / Gemini / GitHub Copilot

The Intent

I used AI to help build the reservation system because it needed to support both normal walk-in reservations and catering/event reservations. I needed help with package rules, package selection limits, deposit calculation, booking summary updates, form validation, and saving reservation data into the database.

The Interaction (Prompting)

I first asked for a reservation form with JavaScript validation. Then I refined the prompt by explaining that the system should include walk-in and catering booking types. I also asked for package rules for different pax sizes, such as 80, 150, 200, 300, and 500 pax, with different deposit amounts and menu selection limits for rice, main dishes, side dishes, and drinks.

The Output & Verification

The AI helped suggest JavaScript logic in `Reservation.html`, including booking type switching, catering package rules, deposit calculation, checkbox selection limits, booking summary updates, and form validation. It also helped with `reserve.php`, which validates form input, checks the logged-in customer, prepares reservation details, and inserts the booking into the database. I verified the reservation system by testing both walk-in and catering modes, submitting valid and invalid forms, checking phone number validation, testing menu selection limits, and confirming that successful submissions appeared in the reservation table.

Human Refinement

I manually changed the reservation code to meet my own project preferences. I edited the package names, package prices, deposit amounts, pax options, menu choices, form labels, validation messages, and summary display. I also changed the JavaScript behavior so the form showed and hid the correct sections based on the selected booking type. On the backend, I adjusted the PHP field names, database column mapping, session/customer handling, and insert logic so it matched the real `reservation` table. The final code was customized for the Munch catering concept instead of using the AI's general reservation example.

4. Authentication and Role-Based Navigation

Project Component: Login Protection, Role Redirects, Customer/Staff/Owner Access Control

AI Tool Used: ChatGPT / Gemini / GitHub Copilot

The Intent

I used AI to help organize login protection for different roles in the system: customer, staff, and owner. The project needed protected pages and buttons to check whether a user was logged in and redirect them to the correct login page if not.

The Interaction (Prompting)

I asked the AI to help create a reusable JavaScript authentication script for a multi-role restaurant web application. I refined the prompt by explaining that my project had separate login pages and separate PHP login-checking files for customers, staff, and owners.

The Output & Verification

The AI helped suggest logic in ``auth.js`` and ``owner-auth.js`` that checks login status with ``fetch``, handles protected links, opens a role selection modal, redirects users to the correct login page, supports redirect parameters, and handles logout buttons. I verified it by logging out, clicking protected links such as profile, reservation, staff dashboard, and owner dashboard, and checking whether each role was redirected correctly.

Human Refinement

I manually changed the authentication code to fit my preferred navigation flow and real project files. I updated the script to use the actual files such as ``customer-login.html``, ``staff-login.html``, ``owner-login.html``, ``checkLogin.php``, ``checkStaffLogin.php``, and ``checkOwnerLogin.php``. I edited the modal text, role labels, protected button selectors, redirect URLs, and login behavior so the user experience matched the Munch website. I also adjusted the JavaScript logic so it worked with my page structure instead of keeping the AI's generic role-authentication example.

5. Customer Profile and User Information Display

Project Component: Profile Page, User Info Fetching, Profile Update, Profile Picture Upload

AI Tool Used: ChatGPT / Gemini / GitHub Copilot

The Intent

I used AI to help connect the customer profile page with the database. The profile page needed to display customer information, allow updates, handle profile picture upload, and show related information such as recent orders, reservations, and payment details.

The Interaction (Prompting)

I asked for help creating JavaScript that could fetch customer information from PHP and update the profile page dynamically. I refined the prompt by asking for support for form submission, profile updates, image upload, missing data handling, and status messages.

The Output & Verification

The AI helped suggest logic for ``profile.js``, ``displayUserInfo.php``, and ``updateProfile.php``. The code fetches user information, fills the profile page, handles empty values, submits updated details, uploads profile images, and displays success or error messages. I verified it by logging in as a customer, opening the profile page, updating profile details, uploading a profile picture, refreshing the page, and checking the database to confirm that changes were saved.

Human Refinement

I manually edited the AI-assisted profile code to match my actual HTML, CSS, database, and preferred layout. I changed JavaScript selectors, PHP queries, form field names, database column names, upload folder paths, default profile values, and user-facing messages. I also adjusted how profile details were displayed so the page matched the Munch design and did not look like a generic generated profile page. The final implementation was integrated with my own login session and database structure.